

gubot_one — Vollständige Projektanalyse

Gazebo-Setups · Sensoren · Duplikate · Bugs

 Pakete: `src/face_tracker` · `src/gubot_gazebo` · `src/gubot_one` · `src/gubot_one_bringup` · `src/gubot_one_description` · `src/nerf_launch_system`

1. GAZEBO-SETUPS — Übersicht

Das Projekt unterstützt **zwei Gazebo-Varianten**, die parallel existieren:

Merkmal	Gazebo Harmonic / Ignition	Gazebo Classic
ros2_control Plugin	<code>gz_ros2_control/GazeboSimSystem</code>	<code>gazebo_ros2_control/GazeboSystem</code>
Gazebo-Plugin	<code>gz_ros2_control-system (gz_ros2_control)</code>	<code>libgazebo_ros2_control.so</code>
Spawn-Tool	<code>ros_gz_sim create</code>	<code>gazebo_ros spawn_entity.py</code>
Bridge-Paket	<code>ros_gz_bridge</code>	direkte ROS-Topics
XACRO-Datei	<code>ros2_control.xacro</code>	<code>gz_classic_ros2_control_sim.xacro</code>

2. GAZEBO HARMONIC / IGNITION — Detail

2.1 Launch-Einstiegspunkte

Für Ignition/Harmonic existieren **zwei parallele Wege**, die dasselbe tun:

Weg A: gubot_one_bringup/launch/launch_sim.launch.py	← HAUPTPFAD (empfohlen)
Weg B: gubot_gazebo/launch/simulation.launch.py	← alternativer Modulpfad
↳ spawn_robot.launch.py	(ruft gubot_one_bringup auf)
Weg C: gubot_one/launch/launch_sim.launch.py	← DUPLIKAT (veraltet, sollte entfernt werden)

2.2 ros2_control XACRO (Ignition)

Datei: gubot_one_description/description/ros2_control.xacro

- ☒ Hardware-Modus: diffdrive_arduino/DiffDriveArduinoHardware +
nerf_launch_system/NerfSystem
- ☒ Fake-Hardware: mock_components/GenericSystem
- ☒ Sim-Modus: gz_ros2_control/GazeboSimSystem
- ☒ IMU StateInterfaces vollständig (inkl. orientation.x/y/z/w)

Sim-Joints (Ignition): left_wheel_joint , right_wheel_joint , tilt_joint , shooter_joint ,
flywheel_left_joint , flywheel_right_joint , system_ariming_joint , imu_sensor

2.3 Bridge-Konfiguration (Ignition)

Weg A (bringup): Inline-Argumente in launch_sim.launch.py

```
/clock          gz.msgs.Clock → ROS
/scan           gz.msgs.LaserScan → ROS
/camera/image_raw gz.msgs.Image → ROS
/camera/camera_info gz.msgs.CameraInfo → ROS
# /imu_sensor/imu_data ← AUSKOMMENTIERT! Fehler - wird nicht gebridged
```

Weg B (gubot_gazebo): YAML-basierter Bridge in config/robot_bridge.yaml

/imu_sensor/imu_data	gz.msgs.IMU → ROS	← ☒ hier korrekt vorhanden
/camera/image_raw		← ☒ vorhanden
/scan		← ☒ vorhanden

⚠ Problem: In gubot_one_bringup/launch/launch_sim.launch.py ist der IMU-Bridge-Eintrag /imu_sensor/imu_data auskommentiert. Der IMU-Broadcaster sendet Daten, aber sie kommen nicht in ROS2 an. Beim Starten via Weg A (bringup) fehlt daher die IMU-Topic-Verbindung.

3. GAZEBO CLASSIC — Detail

3.1 Launch-Einstiegspunkte

```
gubot_one_bringup/launch/gz_classic_launch_sim.launch.py ← HAUPTPFAD
gubot_one/launch/gz_classic_launch_sim.launch.py          ← DUPLIKAT (veraltet)
gubot_gazebo/launch/gz_classic_launch_sim.launch.py      ← DRITTE KOPIE (unklar genutzt)
```

3.2 KRITISCHER BUG — Falsches URDF für Classic-Mode

`rsp.launch.py` lädt immer `gubot_one_description/description/robot.urdf.xacro`, auch wenn `use_gazebo_classic:=true` übergeben wird.

`robot.urdf.xacro` kennt `use_gazebo_classic` NICHT und includet immer:

- `robot_core.xacro` (Ignition-IMU-Sensor, kein Classic-Plugin)
- `ros2_control.xacro` (Ignition-ros2_control)
- `camera.xacro` (Ignition-Kamera-Topic)

```
gz_classic_launch_sim.launch.py
└─> rsp.launch.py [use_gazebo_classic:=true]
    └─> robot.urdf.xacro          ← ignoriert use_gazebo_classic!
        └─> ros2_control.xacro (sim_mode=true)
            └─> gz_ros2_control/GazeboSimSystem ← FALSCH für Classic!
```

Die Datei `robot_classic.urdf.xacro` existiert, wird aber von keinem Launch-File aufgerufen!

`robot_classic.urdf.xacro` würde korrekt laden:

- `robot_core_classic.xacro` (mit `libgazebo_ros_imu_sensor.so`)
- `gz_classic_ros2_control_sim.xacro` (mit `gazebo_ros2_control/GazeboSystem`)

● **KRITISCH:** Gazebo Classic Simulation wird nie korrekt starten, weil das falsche `ros2_control`-Plugin geladen wird. Das Plugin `gz_ros2_control/GazeboSimSystem` existiert nicht in Gazebo Classic.

3.3 Fix für `rsp.launch.py`

`rsp.launch.py` muss auf `use_gazebo_classic` reagieren:

```
# In rsp.launch.py – Xacro-Datei dynamisch wählen
from launch.substitutions import PythonExpression

xacro_file = os.path.join(pkg_path, "description", "robot.urdf.xacro")
xacro_file_classic = os.path.join(pkg_path, "description", "robot_classic.urdf.xacro")
# Oder use_gazebo_classic in robot.urdf.xacro als Verzweigung einbauen
```

Einfachster Fix: `robot.urdf.xacro` um Classic-Branch erweitern.

4. SENSOREN — Vollständige Analyse

4.1 IMU (ICM-20948)

Aspekt	gubot_one_description/robot_core.xacro	gubot_one/description/robot_core.xacro
IMU Position	xyz="0 0 0.05"	xyz="0 0 0.148"
use_gazebo_classic	✗ nicht vorhanden	✓ vorhanden
Classic-Plugin	✗ fehlt	✓ libgazebo_ros_imu_sensor.so
Ignition-Topic	✓ imu_sensor/imu_data	✓ conditional

● **DUPLIKAT + INKONSISTENZ:** Zwei verschiedene robot_core.xacro mit unterschiedlicher IMU-Position (0.05 vs 0.148). Der aktive rsp.launch.py nutzt gubot_one_description — dort fehlt das Classic-Plugin.

IMU in den Launches:

- ✓ Ignition (bringup): imu_broadcaster_spawner + imu_filter_madgwick_node vorhanden
- ✓ gubot_gazebo: imu_filter_node mit config-Datei vorhanden
- ✗ Classic (gz_classic_launch_sim.launch.py): imu_filter_madgwick_node **FEHLT**

4.2 Kamera (RGB)

Datei: gubot_one_description/description/camera.xacro

```
<sensor name="camera" type="camera">
  <topic>camera/image_raw</topic> <!-- Ignition-Style -->
</sensor>
```

● **BUG — Classic-Kamera:** camera.xacro verwendet Ignition-Syntax (nur <topic>). Für Gazebo Classic wird libgazebo_ros_camera.so benötigt. robot_classic.urdf.xacro includes camera.xacro — die Kamera wird in Classic nie publizieren.

Kamera-Position: chassis → xyz="0.276 0 0.181" mit Neigung rpy="0 0.18 0" (ca. 10°)

Auflösung: 640×480, FOV 1.089 rad (ca. 62°), Update-Rate 10 Hz

4.3 Tiefenkamera (Depth Camera)

Datei: gubot_one_description/description/depth_camera.xacro — **existiert**

⚠ **DEAKTIVIERT:** In robot.urdf.xacro auskommentiert: <!-- include filename="depth_camera.xacro" -->
Die Datei ist bereit, aber nicht aktiviert.

4.4 LIDAR

Datei: gubot_one_description/description/lidar.xacro — **existiert**

⚠️ **DEAKTIVIERT:** In `robot.urdf.xacro` auskommentiert: `<!-- <xacro:include filename="lidar.xacro" /> -->`
Obwohl die Karte-Funktionalität (SLAM) konfiguriert ist (`mapper_params_online_async.yaml1`), fehlt der Lidar im Modell.

Bridge in `launch_sim.launch.py` : `/scan` ist gebridged → funktioniert nicht ohne Lidar in der URDF.

4.5 Face-Link / "Augen"

Datei: `gubot_one_description/description/face.xacro`

- ✅ Definiert `face_link` mit zwei "Augen"-Zylindern und rotem Mund-Zylinder (rein visuell)
- Kein Sensor, kein Kamera-Plugin
- Der echte Face-Tracker nutzt die `camera_link` des `camera.xacro`

4.6 Gesichtserkennung (`face_tracker`)

Implementierte Python-Module:

- `detect_face.py` — Gesichtserkennung via OpenCV/face_recognition
- `follow_face.py` — Kopf-/Kamerasteuerung zum Verfolgen
- `fire_at_face.py` — Auslösen des Nerf-Launchers bei erkanntem Gesicht
- `process_image.py` — Bildverarbeitung
- `register_face.py` — Gesichtsregistrierung
- `fake_face_publisher.py` — Fake-Gesichtspublisher für Tests
- `udp_cam_sender/receiver.py` — UDP-Kamerastream (Windows → WSL2)

Launch-Dateien:

- `face_tracker.launch.py` — Standard (echte Kamera)
- `face_tracker_sim.launch.py` — Simulation
- `face_tracker_udp.launch.py` — UDP-Stream

Params:

- `face_tracker_params_robot.yaml1` (`gubot_one_bringup`) ✅
- `face_tracker_params_sim.yaml1` (`gubot_one_bringup`) ✅
- `face_tracker_params.yaml1` (`face_tracker/config`) ✅

5. NERF-LAUNCHER — Analyse

5.1 URDF-Dateien im nerf_launch_system

```
description/urdf/
├─ launcher.urdf.xacro          ← Hauptdatei (Ignition-kompatibel)
├─ gz_classic_launcher.urdf.xacro
├─ launcher_gazebo_classic.urdf.xacro
├─ ros2_control.xacro           ← Sim-ros2_control (Ignition)
├─ gz_classic_ros2_control.xacro ← Classic-ros2_control
└─ ros2_control_hardware.xacro  ← Echte Hardware
```

5.2 KRITISCHER BUG — Flywheel-Joints als FIXED definiert

In `launcher.urdf.xacro` sind die Flywheel-Joints **FIXED** (nicht beweglich):

```
<joint name="flywheel_right_joint" type="fixed">  ← FIXED!
<joint name="flywheel_left_joint" type="fixed">   ← FIXED!
```

In `ros2_control.xacro` (Ignition sim) werden sie aber als velocity-kontrolliert deklariert:

```
<joint name="flywheel_left_joint">
  <command_interface name="velocity"/>
  <state_interface name="velocity"/>
```

●

KRITISCH: Fixed-Joints können keine velocity-Kommandos empfangen. Der Controller-Manager wird beim Spawnen des flywheel-Controllers fehler werfen oder die Controller werden nicht aktivierbar sein. In `launcher.urdf.xacro` müssen `flywheel_left_joint` und `flywheel_right_joint` auf `type="continuous"` umgestellt werden.

5.3 Joint-Namen-Inkonsistenz

Joint	ros2_control.xacro (HW)	gz_classic_ros2_control.xacro (HW)	launcher.urdf.xacro
tilt_joint	✓	✓	✓
shooter_joint	✓	✗ nicht vorhanden	✓ (continuous)
system_arming_joint	✓	✓	✓ (revolute)
trigger_joint	✗	✓	✗ nicht definiert!
dart_pusher_joint	✗	✓	✗ nicht definiert!
flywheel_left_joint	✓ (sim)	✓	⚠ (fixed)
flywheel_right_joint	✓ (sim)	✓	⚠ (fixed)

● **BUG:** gz_classic_ros2_control.xacro (Hardware-Branch) referenziert trigger_joint und dart_pusher_joint, die in launcher.urdf.xacro nicht existieren. Diese Joints fehlen komplett. Das macht die Classic-Hardware-Konfiguration ungültig.

5.4 nerf_launch_system Referenzfehler (gubot_one)

src/gubot_one/description/gz_classic_ros2_control.xacro Zeile ~155:

```
<parameters>$(find gubot_one)/config/my_controllers.yaml</parameters>  
<parameters>$(find gubot_one)/config/gaz_ros2_ctl_use_sim.yaml</parameters>
```

Diese Config-Dateien liegen in gubot_one_bringup/config/ , **nicht** in gubot_one/config/ ! Beim Ausführen über den gubot_one -Pfad gibt es einen Fehler beim Laden der Controller.

6. VOLLSTÄNDIGE DUPLIKATE-LISTE

6.1 XACRO-Dateien

Datei	Kopie 1	Kopie 2	Unterschied
gz_classic_ros2_control.xacro	gubot_one_description/description/	gubot_one/description/	Config-Pfad (gubot_one_brin vs. gubot_one)
robot_core.xacro	gubot_one_description/description/	gubot_one/description/	IMU-Z (0.05 vs 0.148), Classic-Plugin

6.2 Launch-Dateien

Datei	Kopie 1	Kopie 2	Kopie 3
launch_sim.launch.py	gubot_one_bringup/launch/	gubot_one/launch/	—
gz_classic_launch_sim.launch.py	gubot_one_bringup/launch/	gubot_one/launch/	gubot_gazebo/launch/

6.3 World-Dateien

Datei	Kopie 1	Kopie 2
obstacles.world	gubot_gazebo/worlds/	nerf_launch_system/worlds/
empty.world	gubot_gazebo/worlds/	nerf_launch_system/worlds/

6.4 Config-Dateien

Datei	Kopie 1	Kopie 2
gazebo_params.yaml	gubot_one_bringup/config/	nerf_launch_system/config/

6.5 Verwaiste Dateien (Orphans)

Datei	Problem
gubot_one_description/description/robot_classic.urdf.xacro	Existiert, wird von keinem Launch aufgerufen
gubot_one_description/description/robot_core_bak.xml	Backup, sollte gelöscht oder versioniert werden
gubot_one_description/description/gazebo_control.xacro	libgazebo_ros_diff_drive.so Plugin — wird in keinem URDF inkludiert (Classic diff-drive außerhalb ros2_control)

7. ZUSAMMENFASSUNG — Priorisierte Fehlerliste

● KRITISCH — Muss behoben werden

#	Problem	Datei	Lösung
C1	Classic-Simulation lädt falsches ros2_control-Plugin	rsp.launch.py	use_gazebo_classic_robot.urdf.xacro auswerten und robot_classic.urdf laden
C2	Flywheel-Joints sind type="fixed" aber als velocity-Controller deklariert	launcher.urdf.xacro	Auf type="continuous" ändern
C3	trigger_joint und dart_pusher_joint in Classic-HW-xacro referenziert, aber im URDF nicht definiert	gz_classic_ros2_control.xacro	Joints angleichen aus URDF ergänzen
C4	Config-Pfad falsch in gubot_one/description/gz_classic_ros2_control.xacro	gubot_one/description/	\$(find gubot_one) \$(find gubot_one_bringup)

⚠ WICHTIG — Funktionsprobleme

#	Problem	Datei	Lösung
W1	IMU-Bridge auskommentiert in bringup launch_sim	launch_sim.launch.py	Zeile /imu_sensor/imu_data@... einkommentieren
W2	Kamera funktioniert nicht in Gazebo Classic	camera.xacro	camera_classic.xacro mit libgazebo_ros_camera.so erstellen
W3	IMU-Filter fehlt in Classic-Launch	gz_classic_launch_sim.launch.py	imu_filter_madgwick_node hinzufügen
W4	IMU-Position unterschiedlich (0.05 vs 0.148 Z)	beide robot_core.xacro	Einen kanonischen Wert festlegen
W5	/scan Bridge aktiv, aber Lidar auskommentiert	robot.urdf.xacro	Lidar aktivieren oder Bridge entfernen

📄 AUFRÄUMEN — Duplikate / Verwaiste Dateien

#	Aktion	Datei
D1	Löschen	gubot_one/description/robot_core.xacro (Duplikat)
D2	Löschen	gubot_one/description/gz_classic_ros2_control.xacro (Duplikat)
D3	Löschen	gubot_one/launch/launch_sim.launch.py (Duplikat)

D4	Löschen	gubot_one/launch/gz_classic_launch_sim.launch.py (Duplikat)
D5	Aktivieren oder löschen	gubot_one_description/description/robot_core_bak.xml
D6	Prüfen/Aktivieren	gubot_one_description/description/lidar.xacro
D7	Prüfen/Aktivieren	gubot_one_description/description/depth_camera.xacro
D8	Löschen	nerf_launch_system/worlds/obstacles.world + empty.world
D9	Löschen	nerf_launch_system/config/gazebo_params.yaml
D10	Einbinden	gubot_one_description/description/robot_classic.urdf.xacro in rsp.launch.py

8. EMPFOHLENE PAKETSTRUKTUR

```

src/
├─ gubot_one_description/      ← EINZIGE Quelle für URDF/XACRO
│   └─ description/
│       ├── robot.urdf.xacro    (Ignition-Hauptdatei + use_gazebo_classic Branch)
│       ├── robot_core.xacro    (eine Datei, mit Classic-Branch via xacro:if)
│       ├── ros2_control.xacro  (Ignition sim + Hardware)
│       ├── gz_classic_ros2_control_sim.xacro (Classic sim only)
│       ├── camera.xacro        (Ignition)
│       ├── camera_classic.xacro (NEU - Classic mit libgazebo_ros_camera.so)
│       ├── lidar.xacro         (aktivieren!)
│       └─ nerf_launcher.xacro
│
├─ gubot_one_bringup/          ← EINZIGE Quelle für Launch-Dateien
│   └─ launch/
│       ├── launch_sim.launch.py (Ignition)
│       ├── gz_classic_launch_sim.launch.py (Classic)
│       ├── launch_robot.launch.py (echte Hardware)
│       └─ rsp.launch.py         (URDF nach use_gazebo_classic wählen)
│
├─ gubot_gazebo/               ← Nur Gazebo-spezifische Konfiguration
│   ├── worlds/                (obstacles.world, empty.world)
│   └─ config/                 (gz_bridge.yaml, imu_filter.yaml)
│
├─ nerf_launch_system/         ← Hardware-Interface + Firmware
├─ face_tracker/               ← Gesichtserkennung + Schusssteuerung
└─ gubot_one/                  ← NUR package.xml + .repos Dateien (kein URDF/Launch)

```

✅ Was bereits gut funktioniert:

- Vollständige Gazebo Ignition/Harmonic Pipeline (gubot_gazebo) mit OGRE, WSLg-Optimierungen, YAML-Bridge
- IMU mit Madgwick-Filter in Ignition-Launches
- Nerf-Hardware-Interface (nerf_launch_system) mit C++ HW-Plugin, PlatformIO-Firmware

- Face-Tracker-Pipeline (UDP-Kamera, Erkennung, Verfolgung, Schussauslösung)
- Controller-Konfiguration für tilt/shooter/arming vollständig
- Nav2-Parameter und SLAM-Konfiguration vorhanden
- Docker-Setup für Hardware und Simulation